

Encryption and Schema Binding Option in SQL Server Functions

In this article, I am going to discuss the use of **Encryption and Schema Binding Option in SQL Server Functions** with Examples. Please read our previous article where we discussed [Multi-Statement Table Valued Function in SQL Server](#). While creating a user-defined function, we can specify the With Encryption and With SchemaBinding Options. At the end of this article, you will understand when and how to use With Encryption and With SchemaBinding Options in SQL Server.

WITH ENCRYPTION OPTION in SQL Server User-Defined Function:

If you want to encrypt the text of a Function then you need to use the **With Encryption Option** which is provided by the SQL Server while creating the Function. Once you create the function using the “**With Encryption**” option then we cannot view the text of the function using the **sp_helptext** system stored procedure. If we try to view the text using the sp_helptext system stored procedure, then we will get a message stating ‘**the text for the object is encrypted**’.

Example: With Encryption Option in SQL Server Function

Let us understand the use of with encryption option with one example. We are going to use the following Employee table.

ID	Name	Gender	DOB	DeptID
1	Pranaya	Male	1996-02-29 10:53:27.060	1
2	Priyanka	Female	1995-05-25 10:53:27.060	2
3	Anurag	Male	1995-04-19 10:53:27.060	2
4	Preety	Female	1996-03-17 10:53:27.060	3
5	Sambit	Male	1997-01-15 10:53:27.060	1
6	Hina	Female	1995-07-12 10:53:27.060	2

Please use the below SQL Script to create and populate the Employee table with sample data

```
-- Create Employee Table
CREATE TABLE Employee
(
    ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Gender VARCHAR(50),
    DOB DATETIME,
    DeptID INT
)
GO
-- Populate the Employee Table with test data
INSERT INTO Employee VALUES(1, 'Pranaya', 'Male', '1996-02-29 10:53:27.060', 1)
```

```

INSERT INTO Employee VALUES(2, 'Priyanka', 'Female', '1995-05-25 10:53:27.060',
INSERT INTO Employee VALUES(3, 'Anurag', 'Male', '1995-04-19 10:53:27.060', 2)
INSERT INTO Employee VALUES(4, 'Preety', 'Female', '1996-03-17 10:53:27.060', 3)
INSERT INTO Employee VALUES(5, 'Sambit', 'Male', '1997-01-15 10:53:27.060', 1)
INSERT INTO Employee VALUES(6, 'Hina', 'Female', '1995-07-12 10:53:27.060', 2)
GO

```

Example: SQL Server Function without using the With Encryption option

Now let's create an Inline Table-Valued Function without using the With Encryption option. The following example is without using the With Encryption option.

```

-- Function without with encryption option
CREATE FUNCTION fn_GetEmployeeDetailsById
(
    @ID INT
)
RETURNS TABLE
AS
RETURN (SELECT Name, Gender, CAST(DOB AS DATE) AS DOB
        FROM Employee
        WHERE ID = @ID)

```

Now let's view the text of the above SQL Server Function using the sp_helptext system stored procedure as shown below.

[sp_helptext fn_GetEmployeeDetailsById](#)

When we execute the above statement it will give us the following output which is nothing but the CREATE Function Statement of the fn_GetEmployeeDetailsById function.

Text
CREATE FUNCTION fn_GetEmployeeDetailsById
(
@ID INT
)
RETURNS TABLE
AS
RETURN (SELECT Name, Gender, CAST(DOB AS DATE) ...
FROM Employee
WHERE ID = @ID)

Example: SQL Server Function using With Encryption option

Now let's alter the [fn_GetEmployeeDetailsById](#) function to use the With Encryption option as shown below.

```

-- Function with encryption option
ALTER FUNCTION fn_GetEmployeeDetailsById
(

```

```

    @ID INT
)
RETURNS TABLE
WITH Encryption
AS
RETURN (SELECT Name, Gender, CAST(DOB AS DATE) AS DOB
        FROM Employee
        WHERE ID = @ID)

```

Now let's try to view the text of the above SQL Server Function using the sp_helptext system stored procedure as shown below.

sp_helptext fn_GetEmployeeDetailsById

When we execute the above statement you will get a message stating **'The text for object 'fn_GetEmployeeDetailsById' is encrypted.'**

With SCHEMABINDING Option in SQL Server User-Defined Function:

The SCHEMABINDING option specifies that the function is bound to the database objects that it references. So, when the SCHEMABINDING option is specified the database object cannot be modified in any way that would affect the function definition. The function definition itself must first be modified or dropped to remove dependencies on the object that is to be modified.

Suppose we create a function that depends on the Employee table and after creating the function if we modify the table from the database while the function is referencing that table. After modification of the table if we execute the function it will give an error message that is **"Invalid Object Name Employee"**.

Example: SQL Server Function without using the With Schemabinding option

Let's Create a Function without using the "With Schemabinding" option by executing the below script.

```

-- Function without SCHEMABINDING option
CREATE FUNCTION fn_GetEmployeesByGender
(
    @Gender INT
)
RETURNS TABLE
AS
RETURN (SELECT Name, Gender, CAST(DOB AS DATE) AS DOB
        FROM Employee
        WHERE Gender = @Gender)

```

In the above example, the function **fn_GetEmployeesByGender** depends on the table **Employee**. Let's delete the Employee table by executing the below DROP table statement.

DROP TABLE Employee

So, here we are able to delete the table Employee, while the function **"fn_GetEmployeesByGender"** is still referencing it. Once we delete the table, let's try to execute the function as shown below.

SELECT * FROM dbo.fn_GetEmployeesByGender('Male')

When we execute the above SQL Statement, it will give us the below error.

```
Msg 208, Level 16, State 1, Procedure fn_GetEmployeesByGender, Line 28
Invalid object name 'Employee'.
Msg 4413, Level 16, State 1, Line 20
Could not use view or function 'dbo.fn_GetEmployeesByGender' because of binding errors.
```

How to prevent this?

To prevent this, we need to use the SCHEMABINDING option while creating the function and the important thing we need to remember is that we need to specify the two-part name of the table while we are using the SCHEMABINDING option.

Let's create and populate the Employee table using the script provided at the beginning of this article. Once you create and populate the Employee table, let's alter the Function to use the SCHEMABINDING as shown below.

```
-- Function with SCHEMABINDING option
ALTER FUNCTION fn_GetEmployeesByGender
(
    @Gender INT
)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN (SELECT Name, Gender, CAST(DOB AS DATE) AS DOB
        FROM dbo.Employee
        WHERE Gender = @Gender)
```

Now let's try to delete the Employee table by executing the below DROP table statement.

DROP TABLE Employee

When we execute the above statement, it will give us the below error.

```
Msg 3729, Level 16, State 1, Line 20
Cannot DROP TABLE 'Employee' because it is being referenced by object 'fn_GetEmployeesByGender'.
```

If required, then we can use both the “WITH ENCRYPTION” and “WITH SCHEMABINDING” option at the same time as shown below.

```
CREATE FUNCTION fn_GetEmployeesByGender
(
    @Gender INT
)
RETURNS TABLE
WITH SCHEMABINDING, ENCRYPTION
AS
RETURN (SELECT Name, Gender, CAST(DOB AS DATE) AS DOB
        FROM dbo.Employee
        WHERE Gender = @Gender)
```

In the next article, I am going to discuss the [Deterministic and Non-Deterministic Functions in SQL Server](#) with Examples. Here, in this article, I try to explain the **Encryption and Schema Binding Option in SQL Server Functions** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.